

Lab Report 6: Boundary Fill Algorithm

Name: _____ (Fill in your name)

Lab Program Number: 06

Date: _____ (Fill in your date)

Title

Understanding & Implementing Boundary Fill Algorithm

Theory

The Boundary Fill Algorithm is a fundamental technique in computer graphics used to fill closed regions with a specified color. It starts from a seed point inside the region and recursively colors all connected pixels that are not part of the boundary. The algorithm comes in two variants: **4-connected** (which considers up, down, left, and right neighbors) and **8-connected** (which also includes diagonal neighbors). This method is particularly useful for interactive graphics applications where regions need to be filled after being outlined by the user.

Key Concepts:

- **Seed Point:** Starting point inside the region to be filled
 - **Boundary Color:** Color of the region's outline
 - **Fill Color:** Color to fill the region with
 - **4-Connected:** Fills only horizontal and vertical neighbors
 - **8-Connected:** Fills all 8 surrounding pixels (including diagonals)
-

Advantages

- Simple to implement and understand
 - Fills precisely within a defined boundary
 - Effective for simple, bounded shapes
 - Straightforward to debug and use
-

Disadvantages

- Limited to shapes with well-defined boundaries
 - May cause stack overflow with deep recursion
 - Not ideal for complex or irregular shapes
 - Can be inefficient for large areas
-

Algorithm Steps

Boundary Fill Algorithm (4-connected):

Step 1: Start

Initialize fill color, boundary color, and seed point (x, y)

Step 2: Check the current pixel

Get the color of pixel at (x, y)

Step 3: Update pixel color

If pixel is not boundary color and not already filled, set it to fill color

Step 4: Recursively apply to 4-connected neighbors

Process right, left, bottom, and top neighbors

Step 5: Repeat process

Continue until no more pixels need filling

Step 6: Termination

Stop when all reachable pixels within boundary are filled

Boundary Fill Algorithm (8-connected):

Step 1: Start

Initialize fill color, boundary color, and seed point (x, y)

Step 2: Check the current pixel

Get the color of pixel at (x, y)

Step 3: Update pixel color

If pixel is not boundary color and not already filled, set it to fill color

Step 4: Recursively apply to 8-connected neighbors

Process all eight directions including diagonals

Step 5: Repeat process

Continue until no more pixels need filling

Step 6: Termination

Stop when all reachable pixels within boundary are filled

Code

```
c
```

```
#include <stdio.h>
#include <graphics.h>
```

```
// Function to draw a triangle using the line() function
```

```
void drawTriangle(int x1, int y1, int x2, int y2, int x3, int y3) {
    line(x1, y1, x2, y2);
    line(x2, y2, x3, y3);
    line(x3, y3, x1, y1);
}
```

```
// 4-connected boundary fill algorithm
```

```
void boundaryFill4(int x, int y, int fillColor, int boundaryColor) {
    // Check if current pixel is within screen bounds
    if (x < 0 || x >= getmaxx() || y < 0 || y >= getmaxy()) {
        return;
    }
    // Get the current color of the pixel
    int currentColor = getpixel(x, y);

    // If the pixel is not the boundary and not already filled, proceed
    if (currentColor != boundaryColor && currentColor != fillColor) {
        putpixel(x, y, fillColor); // Fill the pixel

        // Recursively call for 4-connected neighbors
        boundaryFill4(x + 1, y, fillColor, boundaryColor); // Right
        boundaryFill4(x - 1, y, fillColor, boundaryColor); // Left
        boundaryFill4(x, y + 1, fillColor, boundaryColor); // Bottom
        boundaryFill4(x, y - 1, fillColor, boundaryColor); // Top
    }
}
```

```
// 8-connected boundary fill algorithm
```

```
void boundaryFill8(int x, int y, int fillColor, int boundaryColor) {
    if (x < 0 || x >= getmaxx() || y < 0 || y >= getmaxy()) {
        return;
    }
    int currentColor = getpixel(x, y);
    if (currentColor != boundaryColor && currentColor != fillColor) {
        putpixel(x, y, fillColor);

        // Recursively call for all 8-connected neighbors
        boundaryFill8(x + 1, y, fillColor, boundaryColor); // Right
        boundaryFill8(x - 1, y, fillColor, boundaryColor); // Left
```

```

    boundaryFill8(x, y + 1, fillColor, boundaryColor); // Bottom
    boundaryFill8(x, y - 1, fillColor, boundaryColor); // Top
    boundaryFill8(x + 1, y + 1, fillColor, boundaryColor); // Bottom-Right
    boundaryFill8(x - 1, y + 1, fillColor, boundaryColor); // Bottom-Left
    boundaryFill8(x + 1, y - 1, fillColor, boundaryColor); // Top-Right
    boundaryFill8(x - 1, y - 1, fillColor, boundaryColor); // Top-Left
}
}

```

```

int main() {
    int gd = DETECT, gm;
    initgraph(&gd, &gm, ""); // Initialize graphics mode

    // Draw a triangle as the boundary for 4-connected fill
    int x1 = 100, y1 = 100;
    int x2 = 250, y2 = 300;
    int x3 = 400, y3 = 100;
    drawTriangle(x1, y1, x2, y2, x3, y3);

    // Define colors
    int boundaryColor = WHITE; // Color of the drawn triangle
    int fillColor1 = GREEN; // Color for 4-connected fill

    // Get a seed point inside the triangle (centroid approximation)
    int seedX = (x1 + x2 + x3) / 3;
    int seedY = (y1 + y2 + y3) / 3;

    printf("Filling triangle with 4-connected algorithm...\n");
    boundaryFill4(seedX, seedY, fillColor1, boundaryColor);
    getch(); // Wait to see the 4-connected fill

    cleardevice(); // Clear screen for next demonstration

    // Draw a rectangle for 8-connected fill demonstration
    printf("Drawing rectangle for 8-connected fill...\n");
    rectangle(150, 150, 350, 350);

    int fillColor2 = RED; // Color for 8-connected fill
    printf("Filling rectangle with 8-connected algorithm...\n");
    boundaryFill8(250, 250, fillColor2, WHITE); // Seed at center

    getch();
    closegraph(); // Close graphics mode
}

```

```
return 0;  
}
```

Output

(Describe or attach your output here)

Example:

The program first displays a triangle filled with green using the 4-connected method. After pressing a key, it shows a rectangle filled with red using the 8-connected method.

Visual demonstration:

- **4-connected fill:** Fills only through direct horizontal/vertical paths
- **8-connected fill:** Can fill through diagonal connections as well

The difference becomes especially visible when boundaries have small gaps - 8-connected fill may "leak" through diagonal gaps that 4-connected fill would stop at.

Conclusion

This lab successfully implemented both 4-connected and 8-connected Boundary Fill Algorithms. The practical exercise demonstrated how these algorithms work to fill closed regions starting from a seed point.

Key findings:

- The **4-connected version** is more conservative and suitable for solid boundaries with no diagonal gaps
- The **8-connected version** can fill through diagonal connections but may "leak" through imperfect boundaries
- Both algorithms use **recursion**, which can lead to stack overflow for large regions
- The **boundary color** serves as the stopping condition for the fill operation

Limitations observed:

- Potential for stack overflow with large regions due to deep recursion
- Inefficient memory usage compared to iterative approaches
- Sensitive to boundary continuity

Possible improvements:

- Implement iterative version using explicit stack/queue
- Add bounds checking optimization
- Consider using scan-line based filling for large areas

This lab provided valuable hands-on experience with region filling techniques fundamental to computer graphics and image processing applications such as paint programs, image editing tools, and game development.

Instructor's Signature: _____

Date: _____

Marks: _____ / _____